THE COMPLETE DEVELOPER HANDBOOK

Java & Spring Boot Mastery

Zero to Advanced

From Programming Fundamentals to Production-Ready Backend Engineering, Microservices, Cloud Deployment, and Interview Preparation

Java 21 LTS · Spring Boot 4.1 · Maven · PostgreSQL

Contents

Before You Begin 3

Copyright and Disclaimer 3

Versions Used in This Book 4

How to Use This Book 4

Part I — Programming and Computer Science Foundations 6

Chapter 1: Introduction to Programming 7

Chapter 2: Java Architecture 23

Before You Begin

Copyright and Disclaimer

Java & Spring Boot Mastery: The Complete Developer Handbook — Zero to Advanced

Copyright © 2026. All rights reserved.

This book is an original work. Product and project names (Java, OpenJDK, Spring, Spring Boot, Hibernate, PostgreSQL, Apache Kafka, Maven, JUnit, Mockito, Docker, Kubernetes, Git, GitHub, IntelliJ IDEA and others) are trademarks of their respective owners. This book is independent and is not endorsed by, or affiliated with, any of them.

Software changes. Languages, frameworks, libraries and cloud services evolve quickly. Version numbers, configuration properties, commands and screenshots in this book were correct for the versions listed in the compatibility table below when the text was written, but may differ in newer releases. Always consult the current official documentation before relying on any detail in production.

No warranty. The code and advice are provided for learning. They are provided "as is", without warranty of any kind. You are responsible for testing, securing and reviewing any code before using it in a real system, particularly code that handles money, personal data, credentials or safety-critical behaviour.

What has been verified. Every Java program in Parts I–II that is printed with an "Output" block was compiled with `javac -Xlint:all` and executed on JDK 21, and the output shown is the captured output of that run. Where a snippet is pseudocode, a fragment, or an example of a *failing* program, the text says so. Examples that need external systems (databases, brokers, cloud platforms) say which version they target and what was and was not executed.

Versions Used in This Book

Component	Version used	Notes
Java	21 LTS (Temurin 21.0.12)	Primary teaching version. Java 25 LTS is covered as an upgrade reference.
Spring Boot	4.1.x	Latest stable line at time of writing (4.1.1). Boot 4 builds on Spring Framework 7.
Build tool	Maven 3.9.x	Gradle comparisons in Chapter 20.
Database	PostgreSQL	Version stated in the chapter that first uses it.
IDE	IntelliJ IDEA	VS Code alternatives are noted where useful.

How to Use This Book

The book moves from “what is a program?” to “how do I run a fault-tolerant system in production?”. You do not need any prior programming experience. If you already know Java, use the table of contents to jump to the part you need, but read the “Prerequisites” box at the start of each chapter first.

Each chapter follows the same rhythm: a plain-language explanation, *why* the idea exists, a worked example, the mistakes beginners make, best practices, exercises, and interview questions with answers.

Throughout the book you will see these boxes:

> **Tip**
>
> **Tips** are shortcuts and habits that save time.

> **Warning**
>
> **Warnings** describe things that cause bugs, security holes or lost data.

> **Interview Insight**
>
> **Interview Insights** point out how a topic is usually asked about in interviews.

> **Best Practice**
>
> **Best Practices** are the conventions professional teams follow, with the reason behind them.

> **Common Mistake**
>
> **Common Mistakes** show a real error message, its cause, and the fix.

> **Real-World Analogy**
>
> **Real-World Analogies** make an idea familiar. Each one also tells you where the analogy stops being accurate.

All code lives in files whose names are printed above each listing. If you type the examples yourself (strongly recommended), keep the file names exactly as shown.

Part I — Programming and Computer Science Foundations

This part assumes nothing. By the end of it you will have a working Java environment and will be able to write, run and reason about small Java programs: variables, decisions, loops, arrays, strings, methods and recursion.

Chapter 1: Introduction to Programming

Learning objectives. After this chapter you will be able to:

- explain what a program is and how a computer executes it;
- distinguish source code, compilation, interpretation and runtime;
- describe a solution as an algorithm, in pseudocode, and as a flowchart;
- explain where Java fits among programming languages and what a Java backend developer does;
- install JDK 21 and an IDE, and compile and run your first Java programs from both the command line and the IDE.

Prerequisites. None. You need a computer and a willingness to type the examples.

1.1 What Is Programming?

A computer is extremely fast and extremely literal. It will do exactly what you tell it, billions of times per second, and it will not guess what you meant. **Programming** is the activity of writing precise instructions, called a **program**, that tell a computer how to solve a problem.

A *problem* might be small ("add two numbers"), everyday ("show my bank balance"), or enormous ("route millions of food orders to the nearest restaurant"). In every case the work has the same shape:

1. Understand the problem exactly.
2. Break it into small, unambiguous steps.
3. Write those steps in a language the computer can run.
4. Run them, find where reality differs from your intention, and fix it.

Step 4 is not a failure of programming; it *is* programming. Professional developers spend a large share of their time reading code and finding out why it does not do what they expected. This book teaches you to do that systematically.

> **Real-World Analogy**
>
> A program is like a **recipe**. A recipe lists ingredients (data) and steps (instructions) in a fixed order. A cook who follows it produces a dish. Where the analogy stops: a human cook can cope with "add salt to taste". A computer cannot. Every instruction must be exact, and the "cook" never improvises.

1.2 How a Computer Runs Your Program

At the lowest level, a computer has three parts that matter to us:

- The **CPU** (central processing unit) carries out very simple operations: add two numbers, compare two numbers, copy a value, jump to a different instruction.
- **Memory (RAM)** holds the program and the data it is working on, while it runs.
- **Storage** (SSD or disk) keeps files permanently, including your program, when it is not running.

The CPU understands only **machine code**: sequences of binary numbers, each one an instruction such as "add these two registers". Writing machine code by hand is slow and error-prone, and it differs between CPU families (an Intel/AMD x86-64 laptop and an Apple or ARM phone use different instruction sets).

So people invented **high-level languages** such as Java, Python and C. You write readable text, called **source code**, and a tool translates it into something the machine can run. There are two classic ways to translate:

Approach	What happens	Examples	Trade-off
Compilation	A **compiler** translates the whole program ahead of time into a separate file the machine can run.	C, C++, Go, Rust	Fast at run time; you must recompile for each platform.
Interpretation	An **interpreter** reads the source and carries out each instruction while the program runs.	Classic Python, shell scripts	Easy to run anywhere; usually slower.

Java uses a clever mixture of both, which is the topic of Chapter 2. For now, remember this pipeline:

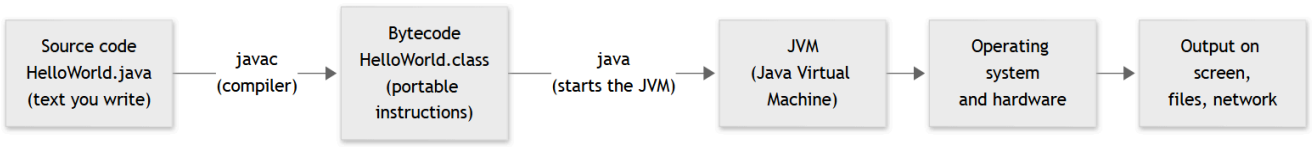


Figure 1.1: From source code to a running Java program

Explanation. You write `.java` files. The Java **compiler** (`javac`) checks them for mistakes and translates them into **bytecode** stored in `.class` files. Bytecode is not tied to one CPU. The **JVM** (Java Virtual Machine), started by the `java` command, loads the bytecode and runs it on whatever machine you are using. This is why the same `.class` file can run on Windows, macOS and Linux. We take the JVM apart in Chapter 2.

Compile time versus run time

Two phrases will appear constantly:

- **Compile time**: the moment `javac` examines your code. Errors found here are *compile-time errors*, such as a missing semicolon or a type mismatch. The program never starts.
- **Run time**: the moment the program is actually executing. Errors here are *runtime errors* (exceptions), such as dividing by zero or opening a file that does not exist.

There is a third category that no tool reports: **logic errors**. The program compiles, runs without crashing, and gives the wrong answer. These are the hardest to find, and they are why we test (Chapter 34).

1.3 Algorithms, Pseudocode and Flowcharts

An **algorithm** is a finite, ordered list of unambiguous steps that solves a problem. "Make tea" is an algorithm; "make good tea" is not, because "good" is not precise.

Before writing any code, experienced developers think in terms of algorithms. Two tools help you do this without worrying about language syntax.

Pseudocode is structured plain English. There is no official grammar; its only rule is that every step must be precise enough that a person could follow it without guessing. Here is an algorithm to find the largest number in a list:

```
PSEUDOCODE: find the largest number in a list
1. Let largest be the first number in the list.
2. For each remaining number in the list:
     a. If the number is greater than largest, set largest to that number.
3. The answer is largest.
```

A **flowchart** draws the same algorithm as a diagram of steps and decisions:

Figure 1.2: Flowchart of the "largest number" algorithm

Explanation. Rounded boxes mark the start and end. Rectangles are actions. Diamonds are yes/no questions. The arrow from "largest = n" back to "Any numbers left?" is a **loop**: the algorithm repeats until the list is exhausted. Notice that the flowchart contains the three building blocks of every program ever written: **sequence** (do steps in order), **selection** (choose based on a condition), and **iteration** (repeat).

Now the same algorithm in Java. Do not worry about understanding every word yet; the comments map each line to the pseudocode. Chapters 3 to 6 explain everything used here.

File: `largest/LargestNumber.java`

```java
/**
 * Finds the largest value in an array using the algorithm
 * described in the chapter: remember the best value seen so far.
 */
public class LargestNumber {

    static int findLargest(int[] numbers) {
        int largest = numbers[0];               // step 1: assume the first is the largest
        for (int i = 1; i < numbers.length; i++) { // step 2: look at every other number
            if (numbers[i] > largest) {         // step 3: is this one bigger?
                largest = numbers[i];           //         if so, remember it
            }
        }
        return largest;                         // step 4: report the answer
    }

    public static void main(String[] args) {
        int[] marks = {42, 77, 15, 98, 63};
        System.out.println("Marks: " + java.util.Arrays.toString(marks));
        System.out.println("Highest mark: " + findLargest(marks));
    }
}
```

Line-by-line.

- `static int findLargest(int[] numbers)` declares a **method** (a named, reusable block of code) that takes an array of integers and returns one integer.
- `int largest = numbers[0];` is step 1 of the pseudocode: start with the first number. Array positions start at 0.
- The `for` loop visits positions 1, 2, 3, … up to the end. This is the "for each remaining number" step.
- The `if` is the decision diamond. `>` means "greater than".
- `return largest;` hands the answer back to whoever called the method.
- `main` is where Java starts running a program. It builds an array called `marks`, prints it, and prints the result of `findLargest(marks)`.

Execution flow. `main` runs first. It calls `findLargest`, passing the five marks. Inside, `largest` begins as 42, then becomes 77 (77 > 42), stays 77 when 15 is checked, becomes 98, and stays 98 when 63 is checked. The method returns 98 to `main`, which prints it.

Output

```
Marks: [42, 77, 15, 98, 63]
Highest mark: 98
```

> **Best Practice**
>
> **Think first, code second.** When a problem is new, write the pseudocode on paper before typing Java. Most beginner "coding" difficulty is really "thinking" difficulty, and it is much cheaper to fix a wrong plan than wrong code.

1.4 Where Java Fits

Java was created at Sun Microsystems in the early 1990s, first released in 1995, and is now stewarded by Oracle together with the open-source **OpenJDK** project and a large community. Its original promise was **"write once, run anywhere"**: compile once to bytecode, run on any machine with a JVM. Three decades later it is still one of the most widely used languages in professional software, especially for server-side systems.

Java's main characteristics:

- **Statically typed.** Every variable has a declared type that the compiler checks. This catches whole classes of mistakes before the program runs.
- **Object-oriented.** Programs are organised as objects that combine data and behaviour (Chapter 7).
- **Automatic memory management.** A **garbage collector** frees memory you no longer use, so you do not manually release it as in C or C++ (Chapter 16).
- **Platform independent.** Bytecode runs on the JVM on any supported operating system.
- **A vast ecosystem.** Libraries and frameworks (Spring, Hibernate, Kafka clients and thousands more) mean you rarely build common pieces from scratch.
- **Strong backward compatibility.** Code written years ago usually still compiles and runs, which is why large organisations trust it.

Typical uses: web back ends and REST APIs, banking and payment systems, large enterprise software, Android applications (historically; Kotlin is now also common), big-data tools, and high-throughput messaging systems.

Java versions and long-term support

Java releases a new version every six months. Most are short-lived. Every few years a version is designated **LTS (Long-Term Support)**, which means vendors maintain it for years. Companies deploy LTS versions. The current LTS line is **8, 11, 17, 21 and 25**. This book teaches on **Java 21** and explains what changed in **Java 25** where it matters.

> **Warning**
>
> Older tutorials often use Java 8 or Java 11. That code still works, but newer features (records, text blocks, switch expressions, pattern matching, virtual threads) make modern Java shorter and safer. Chapter 14 shows the differences side by side.

1.5 What Does a Java Backend Developer Do?

A **backend** developer builds the part of an application that users do not see: the server that stores data, enforces rules and answers requests from web pages and mobile apps.

Imagine ordering food on a phone. The app (front end) shows menus. When you tap "Place order" it sends a request over the internet to a back-end service. That service checks that you are logged in, validates the order, charges the payment, saves the order in a database, tells the restaurant, and replies "Order confirmed". Everything after the tap is back-end work.

Day to day, a Java backend developer typically:

- designs and builds **REST APIs** that other systems call;
- models business rules in code and stores data in **databases**;
- secures APIs with authentication and authorisation;
- writes **automated tests** and reviews teammates' code;
- packages and deploys services with **Docker** and CI/CD pipelines;
- monitors services in production and investigates problems.

The road from zero to job-ready follows the structure of this book:

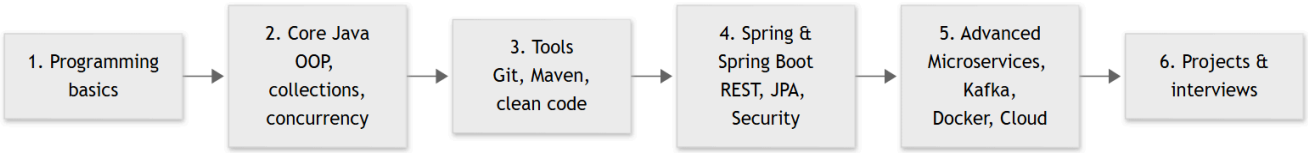


Figure 1.3: Java backend learning roadmap

Explanation. Each stage depends on the previous one. Learners who jump straight to Spring Boot without core Java usually hit a wall: framework error messages assume you understand objects, interfaces, generics and exceptions. Chapter 52 turns this roadmap into a weekly study plan.

1.6 Setting Up Your Development Environment

You need three things: a **JDK**, an **editor or IDE**, and a **terminal**.

Step 1: Install the JDK

The **JDK (Java Development Kit)** contains the compiler (`javac`), the runtime (`java`) and standard tools. Several organisations build free, production-quality OpenJDK distributions. This book was tested with **Eclipse Temurin 21**, available from adoptium.net. Other vendors' builds of Java 21 behave the same for everything in this book.

- **Windows:** download the Temurin 21 installer (`.msi`). In the installer, enable the options to *set* `JAVA_HOME` and *add to PATH*.
- **macOS:** download the `.pkg` installer, or use Homebrew's Temurin cask.
- **Linux:** install your distribution's `openjdk-21-jdk` package, or use Temurin's package repository.

Step 2: Verify the installation

Open a new terminal (PowerShell, Command Prompt, Terminal or a shell) and run:

```
java -version
javac -version
```

On the machine used to write this book, `java -version` printed:

```
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS, mixed mode, sharing)
```

Your exact numbers will differ if you install a different update, but the major version must be `21`. Both commands must work. If `java` works but `javac` does not, you installed only a **JRE** (runtime) and need the full JDK.

Step 3: Install an IDE

An **IDE (Integrated Development Environment)** combines an editor, a compiler, a debugger and project tools. Pick one:

- **IntelliJ IDEA** (JetBrains): the most widely used Java IDE in industry and the one used in this book. JetBrains offers a free edition; check their site for the current editions and licence terms, as these have changed over time.
- **Visual Studio Code** with the *Extension Pack for Java*: a lighter alternative that works well for learning.

> **Tip**
>
> Learn the command line even though the IDE hides it. When something fails on a build server there is no IDE, only a terminal, and you must know what `javac`, `java` and `mvn` actually do.

Step 4: Install Maven (later) and Git

Maven (Chapter 20) and Git (Chapter 19) are needed from Part III onward, and you can install them now: Maven 3.9.x from maven.apache.org, Git from git-scm.com. Verify with `mvn -v` and `git --version`.

1.7 Your First Program: Hello, World

Create a folder called `javabook`, and inside it a file named exactly `HelloWorld.java`:

File: `hello/HelloWorld.java`

```
public class HelloWorld {
    public static void main(String[] args) {
```

```
        System.out.println("Hello, World!");    }     }
```

Line-by-line.

- `public class HelloWorld { ... }` declares a **class**, Java's basic unit of code. The class name must match the file name, including capital letters.
- `public static void main(String[] args)` is the **entry point**. The JVM looks for a method with exactly this signature when starting the program. `public` means callable from anywhere; `static` means it belongs to the class itself, not to an object; `void` means it returns nothing; `String[] args` receives command-line arguments.
- `System.out.println("Hello, World!");` prints text followed by a new line. `System.out` is the standard output stream; `println` is a method on it. Every statement ends with a semicolon.
- Curly braces `{ }` group code into blocks.

Compile and run from the terminal. In the folder that contains the file:

```
javac HelloWorld.java
java HelloWorld
```

`javac` produces `HelloWorld.class`. `java HelloWorld` starts the JVM and runs it (note: you give the *class name*, without `.class`).

Output

```
Hello, World!
```

Execution flow. (1) `javac` parses the file, checks it, and writes bytecode. (2) `java` starts a JVM. (3) The JVM loads `HelloWorld.class`. (4) It finds `main` and executes its one statement. (5) `main` ends, so the program ends.

A shortcut. Since Java 11 you can run a single source file directly, without a separate `javac` step:

```
java HelloWorld.java
```

The launcher compiles the file in memory and runs it. This is handy for experiments, but real projects always use a build tool.

> **Note**
>
> Java 21 includes a *preview* feature for shorter first programs without an explicit class. A final version of that idea ("compact source files and instance `main` methods", JEP 512) arrived in Java 25. This book uses the classic form shown above because it works on every Java version and matches the code you will meet in real projects.

Run it in the IDE. In IntelliJ IDEA: create a new project (Build system: *IntelliJ* or *Maven*, JDK: 21), add a class named `HelloWorld`, type the code, and click the green triangle beside `main`. The *Run* window shows the same output.

1.8 Reading Input: A Slightly Bigger Program

Programs become useful when they respond to data. This program asks for a name and an age:

File: `greeter/Greeter.java`

```
import java.util.Scanner;

public class Greeter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("What is your name? ");
        String name = scanner.nextLine();

        System.out.print("How old are you? ");
        int age = scanner.nextInt();

        System.out.println("Hello, " + name + "! Next year you will be " + (age + 1) + ".");
        scanner.close();
    }
}
```

Key ideas.

- `import java.util.Scanner;` makes the `Scanner` class from Java's standard library available. Libraries are how you reuse other people's tested code.
- `new Scanner(System.in)` creates a scanner that reads from the keyboard (**standard input**).
- `nextLine()` reads a whole line as text (`String`); `nextInt()` reads a whole number (`int`).
- `"Hello, " + name + ...` joins text and values. The parentheses in `(age + 1)` force the addition to happen *before* joining; without them, Java would join the digits "20" and "1" as text and print "201".

A session at the terminal looks like this (what you type follows each prompt):

```
What is your name? Asha
How old are you? 20
Hello, Asha! Next year you will be 21.
```

The final line was produced by the program when it was compiled and run with the input `Asha` and `20`.

File: `average/AverageCalculator.java`

```
public class AverageCalculator {

    /** Returns the average of the given scores, or 0 if there are none. */
    static double average(int[] scores) {
        if (scores.length == 0) {
```

```
            return 0;        }         int total = 0;        for (int score : scores) {
            total += score;        }         return (double) total / scores.length;    }
             public static void main(String[] args) {        int[] scores = {70, 85, 90, 65};
        System.out.println("Average: " + average(scores));
        System.out.println("Empty:   " + average(new int[0]));    }     }
```

This last example combines everything: a method, a loop, a decision (the empty-array guard), and a **cast** `(double) total` that makes the division produce a decimal result instead of dropping the fraction. Without the cast, `total / scores.length` would divide two integers and discard the remainder. We study casting in Chapter 3.

Output

```
Average: 77.5
Empty:   0.0
```

1.9 Common Mistakes and How to Read Error Messages

Beginners often panic at red text. Compiler messages are precise and, once you can read them, very helpful. Always read the **first** error first; later errors are often consequences of it.

Common Mistake

Missing semicolon. The program below forgets the `;` after the `println(...)` call.

```
Broken.java:3: error: ';' expected
        System.out.println("Hello, World!")
                                           ^
1 error
```

How to read it. The message gives the file, the line number (3), what is wrong (`';' expected`), the offending line, and a caret `^` pointing at the spot. *Fix:* add the semicolon.

Common Mistake

Wrong type and a typo. Putting text in an integer variable and misspelling a method name:

```
Broken2.java:3: error: incompatible types: String cannot be converted to int
        int count = "five";
                    ^
Broken2.java:4: error: cannot find symbol
        System.out.printn(count);
                  ^
  symbol:   method printn(int)
  location: variable out of type PrintStream
2 errors
```

Fix: give `count` a number (`int count = 5;`) and spell the method `println`. "Cannot find symbol" almost always means a typo, a missing `import`, or a name used outside the place where it exists.

Common Mistake

File name does not match the public class. Saving `public class HelloWorld` in a file called `Hello.java`:

```
Hello.java:1: error: class HelloWorld is public, should be declared in a file named
HelloWorld.java
public class HelloWorld {
       ^
1 error
```

Fix: rename the file to `HelloWorld.java`, or rename the class. Java is **case-sensitive**: `HelloWorld`, `helloworld` and `Helloworld` are different names.

Common Mistake

Wrong case when running. After a successful compile, `java Helloworld` fails with:

```
Error: Could not find or load main class Helloworld
Caused by: java.lang.NoClassDefFoundError: Helloworld (wrong name: HelloWorld)
```

Fix: run `java HelloWorld`. On Windows, file names are not case-sensitive, so the file can be found while the class inside has a different name, which produces this slightly confusing message.

Common Mistake

`'javac' is not recognized` (Windows) or `command not found` (macOS/Linux). The terminal cannot find the JDK. Either the JDK is not installed, you installed only a JRE, or its `bin` folder is not on the `PATH`. *Fix:* reinstall with "add to PATH" enabled, then **open a new terminal** (old terminals keep the old PATH).

A debugging mindset. When something fails: (1) read the exact message, (2) find the line it points to, (3) look at the line *before* it too, since a missing character often shows up on the next line, (4) change one thing at a time, (5) search for the exact error text only after you have tried to understand it yourself.

1.10 Best Practices from Day One

> **Best Practice**
>
> - **Use meaningful names:** `highestMark` tells a reader more than `x`.
> - **Format consistently.** Indent by four spaces inside each block. IDEs can reformat code for you (IntelliJ: *Ctrl+Alt+L*).
> - **Compile and run often.** Write five lines, run them. Do not write 200 lines and then hunt for the one mistake.
> - **Type the examples; do not paste them.** Your fingers and eyes learn syntax that way.
> - **Keep every program you write** in a Git repository (Chapter 19). Your history of small projects becomes evidence of your skill.

1.11 Exercises

Do these in order. Try before you look anything up.

1. **Hello, you.** Change `HelloWorld` so that it prints your own name on one line and your city on the next line. Compile and run it from the terminal.
2. **Break it on purpose.** Remove the closing brace `}` of `HelloWorld`, then compile. Read the error. Then restore it and introduce two different errors from Section 1.9. Predict each message before compiling.
3. **Pseudocode first.** Write pseudocode, then a flowchart on paper, for an algorithm that decides whether a number is even or odd. (Hint: even numbers leave remainder 0 when divided by 2.) You do not need to code it yet.
4. **Modify the algorithm.** Change `LargestNumber` so that it finds the *smallest* number. Which one line of the logic must change?
5. **Extend the average.** In `AverageCalculator`, add a second method that returns the *highest* score using the approach from `LargestNumber`, and print both. Then explain why `(double) total / scores.length` is not the same as `(double) (total / scores.length)`.
6. **Challenge: trace by hand.** For `findLargest(new int[]{5, 3, 9, 9, 1})`, write a table with the value of `i`, `numbers[i]` and `largest` after each loop iteration. What is the final value, and what does the code do when two numbers tie?

1.12 Interview Questions and Answers

> **Interview Insight**
>
> Chapter 1 topics are rarely asked as "pure theory" in interviews, but interviewers use them to check that your fundamentals are clear. Expect them in the first five minutes of a junior-level screening.

Q1. What is the difference between a compiler and an interpreter? A compiler translates a whole program into another form before it runs; an interpreter executes the source (or an intermediate form) step by step while the program runs. Java does both: `javac` compiles source to bytecode, and the JVM then interprets or just-in-time compiles that bytecode to machine code.

Q2. What is bytecode, and why does Java use it? Bytecode is the instruction set of the Java Virtual Machine, stored in `.class` files. Because it is independent of any physical CPU, one compiled program runs on any platform that has a JVM. This supports Java's "write once, run anywhere" goal.

Q3. What is the difference between the JDK, the JRE and the JVM? The JVM runs bytecode. The JRE is the JVM plus the standard class libraries needed to run programs. The JDK is the JRE plus development tools such as `javac`. Developers install the JDK; modern distributions ship the JDK and no longer emphasise a separate JRE download. (Chapter 2 covers this in depth.)

Q4. What is the entry point of a Java program? The JVM begins execution at `public static void main(String[] args)` in the class you name on the command line. `public` so the JVM can call it from outside, `static` so no object is needed first, and `void` because it returns nothing to the JVM.

Q5. What is the difference between a compile-time error, a runtime error and a logic error? A compile-time error is found by the compiler and stops the program from being built (for example a missing semicolon or a type mismatch). A runtime error occurs while the program runs and typically throws an exception (for example, dividing by zero). A logic error produces a wrong result without any error message and is found only by testing or code review.

Q6. Why is Java called platform independent, and is that fully true? The bytecode is platform independent, but the JVM itself is platform specific: there is a different JVM build for Windows, Linux and macOS. So Java programs are portable; the JVM is not.

Q7. (Scenario) You run `javac HelloWorld.java` and get "class HelloWorld is public, should be declared in a file named HelloWorld.java". What happened and how do you fix it? A public class must live in a file with the same name. The file was probably named differently (for example `Hello.java`). Rename the file or the class so they match, remembering that Java is case-sensitive.

Q8. (Coding) Write a method that returns the largest number in an `int[]`. What happens on an empty array, and how would you handle it? The method in Section 1.3 starts with `numbers[0]`, so an empty array throws `ArrayIndexOutOfBoundsException`. A robust version validates the input first, either by throwing an `IllegalArgumentException("array must not be empty")` or by returning an `OptionalInt`. Time complexity is O(n) because each element is visited once; extra space is O(1).

Follow-up: *How would the code change if the values were `Integer` objects, or if a stream were used?* (We answer this in Chapters 11 and 13.)

1.13 Chapter Summary

- A **program** is an exact list of instructions; **programming** is writing them, running them and fixing the differences between what you meant and what happened.
- Source code is translated for the machine by **compilation** and/or **interpretation**. Java compiles to **bytecode**, which the **JVM** runs.
- An **algorithm** is a finite, precise sequence of steps. Pseudocode and flowcharts let you design algorithms before coding. All algorithms combine sequence, selection and iteration.

- Java is statically typed, object-oriented, garbage-collected and platform independent, with LTS releases 8, 11, 17, 21 and 25. This book uses **Java 21**.
- A backend developer builds servers, APIs, databases, security and deployment pipelines.
- You installed a JDK and an IDE, and you ran programs from both the command line and the IDE.
- Read compiler errors from the first one, understand their structure (file, line, message, caret), and compile often.

1.14 Key Terms and Revision Notes

Term	Meaning
Algorithm	A finite, ordered, unambiguous list of steps that solves a problem
Pseudocode	Informal, structured description of an algorithm
Source code	The human-readable program text (`.java`)
Compiler (`javac`)	Translates source into bytecode
Bytecode	Portable instructions in `.class` files
JVM	The virtual machine that loads and runs bytecode
JDK / JRE	Development kit (with tools) / runtime environment
LTS	Long-Term Support release (8, 11, 17, 21, 25)
Compile-time error	Detected by the compiler; the program does not build
Runtime error	Occurs while running; usually an exception
Logic error	Program runs but gives wrong results
Entry point	`public static void main(String[] args)`
IDE	Integrated Development Environment (IntelliJ IDEA, VS Code)

Quick revision checklist.

- ☐ I can explain source → bytecode → JVM in my own words.
- ☐ I can write a simple algorithm as pseudocode and a flowchart.
- ☐ `java -version` and `javac -version` both show 21 on my machine.
- ☐ I compiled and ran `HelloWorld` from the terminal and from the IDE.
- ☐ I can read a `javac` error message and name the file, line and problem.

Where we go next. Chapter 2 opens the JVM: how class files are loaded, how memory is organised, and why Java is fast even though it runs "interpreted" bytecode.

References for this chapter. Oracle, *The Java Language Specification, Java SE 21 Edition*; OpenJDK project documentation (openjdk.org), including JEP 330 (launch single-file source-code programs) and JEP 512 (compact source files and instance main methods); Eclipse Adoptium (adoptium.net) for Temurin builds; the Java SE release roadmap (oracle.com/java/technologies/java-se-support-roadmap.html) for LTS status.

Chapter 2: Java Architecture

Learning objectives. After this chapter you will be able to:

- distinguish the Java language, the JDK, the JRE and the JVM;
- read simple bytecode with `javap` and explain what a class file contains;
- describe the compile, load, link, initialise and execute lifecycle;
- explain the roles of the interpreter and the JIT compiler;
- name the JVM's main memory areas and say what lives in each;
- explain what "platform independent" really means, and how Java versions and LTS releases work.

Prerequisites. Chapter 1 (you can compile and run a Java program).

2.1 Four Names That Get Confused

Beginners hear "Java", "JDK", "JRE" and "JVM" used almost interchangeably. They are four different things.

- **The Java language** is the set of rules for writing programs: keywords, syntax, types. It is defined by the *Java Language Specification*.
- **The JVM (Java Virtual Machine)** is a program that loads and runs bytecode. Its behaviour is defined by the *Java Virtual Machine Specification*. Many implementations exist; the one in OpenJDK and in Temurin is called **HotSpot**.
- **The JRE (Java Runtime Environment)** is the JVM plus the standard class libraries (`String`, `ArrayList`, `Files`, …) needed to *run* programs.
- **The JDK (Java Development Kit)** is the JRE plus the tools needed to *build* programs: `javac`, `javap`, `jar`, `jshell`, `jlink` and more.

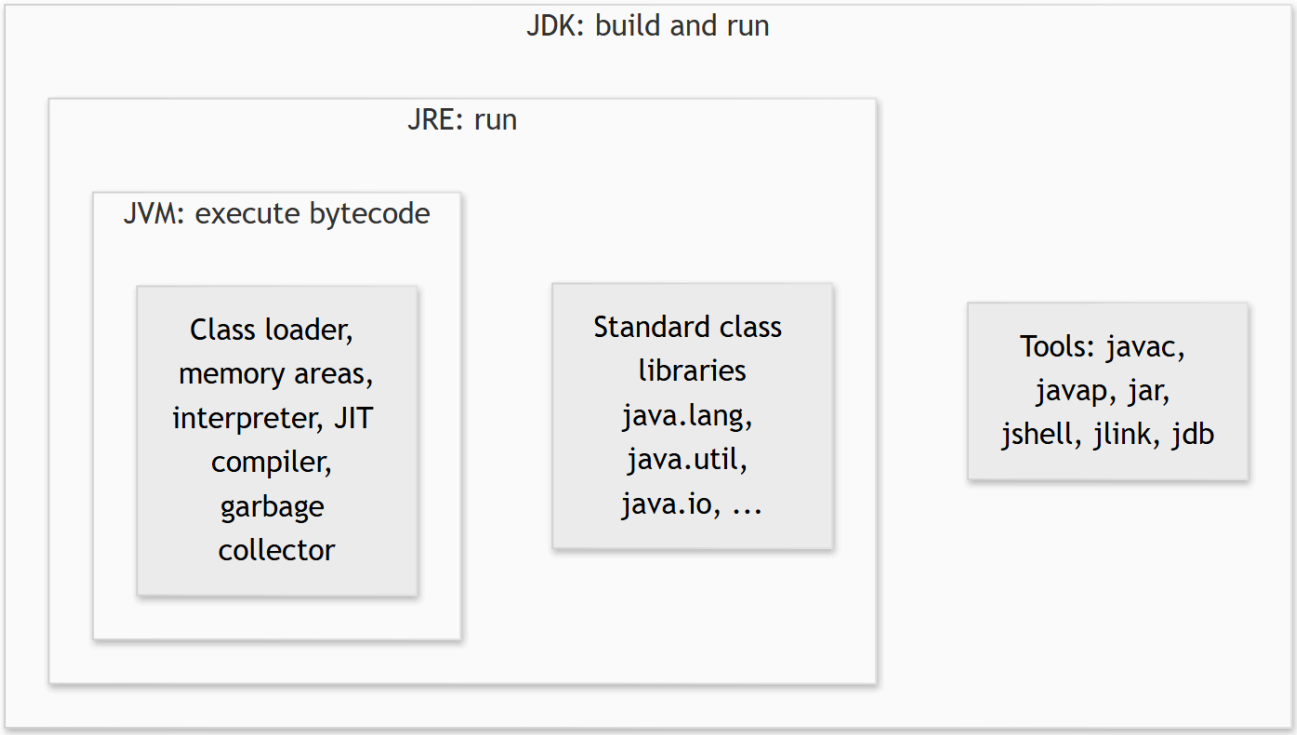


Figure 2.1: JDK, JRE and JVM are nested

Explanation. Each box contains the one inside it. When you only need to run a finished application, the runtime pieces are enough. When you write code, you need the JDK.

Term	Contains	Used by	Key command
JVM	Loader, memory, execution engine, GC	The runtime	`java` starts one
JRE	JVM + class libraries	People who only run programs	`java`
JDK	JRE + development tools	Developers	`javac`, `javap`, `jar`

> **Note**
>
> Since Java 9, the platform is **modular**, and since Java 11 most vendors no longer publish a separate JRE download. Today you install a JDK for development. For deployment, you can use the `jlink` tool to create a small custom runtime that contains only the modules your application needs. In day-to-day work, "install Java" means "install a JDK".

2.2 The Lifecycle of a Java Program

Everything that happens between "I typed code" and "output appears" is a pipeline:

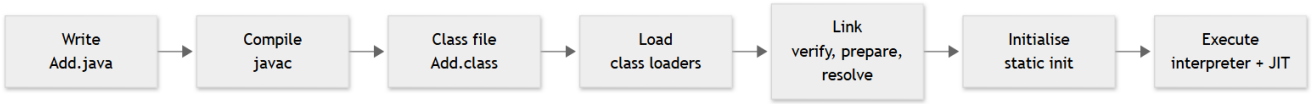


Figure 2.2: Compile-time and run-time stages

The left half (write, compile) happens on the developer's machine, once. The right half (load, link, initialise, execute) happens inside the JVM every time the program runs. We take the stages in order.

2.3 Bytecode and Class Files

The compiler does not produce machine code for a particular processor. It produces **bytecode**, instructions for an imaginary processor, the JVM. The bytecode for each class is stored in a **class file** (`.class`).

A class file is a binary file that always starts with the same four bytes (`CA FE BA BE`, a famous in-joke) and then records the class's name, its parent class, the **constant pool** (a table of names and literals), its fields, and the bytecode of each method. It also stores a **major version number** that says which Java version compiled it: 52 means Java 8, 55 means Java 11, 61 means Java 17, 65 means Java 21 and 69 means Java 25. A JVM refuses to load a class file whose major version is newer than it understands. That is the cause of the common error `UnsupportedClassVersionError` when you run a Java 21 build on a Java 17 runtime.

You can look inside with the JDK's disassembler, `javap`. Here is a tiny class:

File: `bytecode/Add.java`

```
public class Add {
    static int add(int a, int b) {
        int sum = a + b;
        return sum;
    }

    public static void main(String[] args) {
        System.out.println("2 + 3 = " + add(2, 3));
    }
}
```

Compile it and disassemble it with `javac Add.java` and `javap -c -p Add.class` (`-c` shows bytecode, `-p` includes private members):

Output

```
Compiled from "Add.java"
public class Add {
  public Add();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  static int add(int, int);
    Code:
       0: iload_0
       1: iload_1
       2: iadd
       3: istore_2
       4: iload_2
       5: ireturn

  public static void main(java.lang.String[]);
    Code:
       0: getstatic     #7                  // Field java/lang/System.out:Ljava/io/PrintStream;
       3: iconst_2
       4: iconst_3
       5: invokestatic  #13                 // Method add:(II)I
       8: invokedynamic #19,  0             // InvokeDynamic #0:makeConcatWithConstants:
(I)Ljava/lang/String;
      13: invokevirtual #23                 // Method java/io/PrintStream.println:
(Ljava/lang/String;)V
      16: return
}
```

Reading it. The JVM is a **stack machine**: instructions push values onto an **operand stack** and pop them off to compute. For `add(int a, int b)`:

1. `iload_0` pushes local variable 0 (`a`) onto the operand stack; `iload_1` pushes `b`.
2. `iadd` pops two integers, adds them and pushes the result.
3. `istore_2` pops the result into local variable 2 (`sum`).
4. `iload_2` pushes `sum` again and `ireturn` returns the value on top of the stack.

The prefix letter tells you the type: `i` means `int`. In `main`, `iconst_2` and `iconst_3` push the constants 2 and 3, `invokestatic` calls `add`, and `invokevirtual` calls `println`. The string concatenation became an `invokedynamic` instruction: a modern compiler technique (used since Java 9) that lets the JVM choose the most efficient way to build the text. The constructor `Add()` was written by the compiler: if you declare no constructor, it adds a default one that calls `Object`'s constructor.

> **Tip**
>
> You will rarely read bytecode in daily work, but `javap -c` is a superb way to settle arguments about what the compiler really does. For example, it shows whether a `+` in a loop creates new strings (Chapter 5) or what a `switch` compiles into.

2.4 Class Loading, Linking and Initialisation

The JVM does not load every class at start-up. It loads each class **on demand**, the first time the program needs it. For each class, the JVM performs three phases.

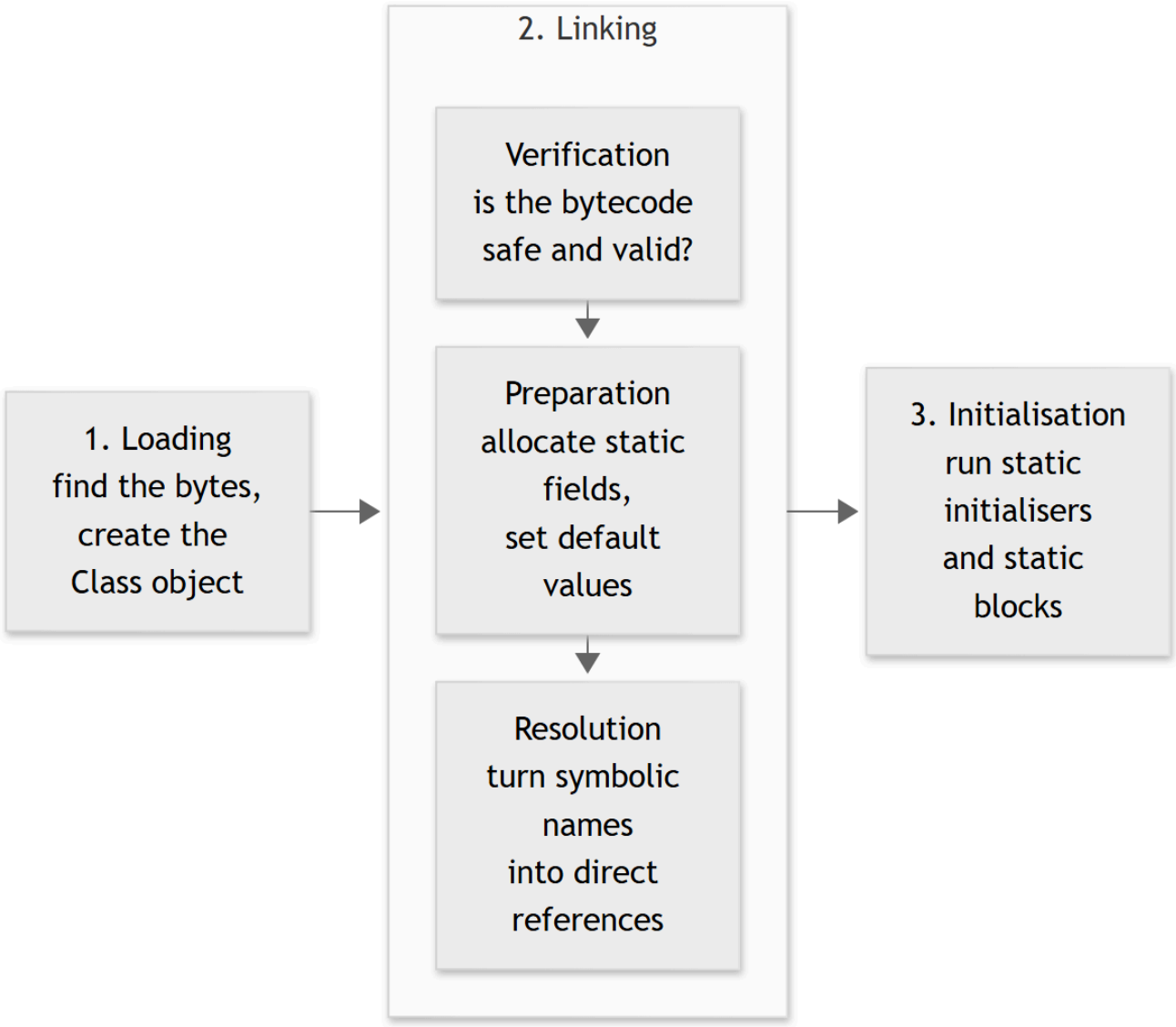


Figure 2.3: Phases a class goes through before use

Explanation.

- **Loading.** A **class loader** reads the class file and creates an in-memory representation of the class.
- **Verification** checks that the bytecode obeys the JVM's rules (correct types on the stack, valid jumps). This is a security feature: it stops damaged or malicious class files from corrupting the JVM.
- **Preparation** reserves memory for `static` fields and gives them default values (`0`, `false`, `null`).
- **Resolution** replaces symbolic references ("the method `add` in class `Add`") with direct references. This is often done lazily, the first time the reference is used.
- **Initialisation** runs the class's `static { ... }` blocks and static field initialisers, in the order they appear in the source. A class is initialised the first time it is actively used: when an instance is created, a static method is called, or a non-constant static field is read or written.

Class loaders

A **class loader** is the object that finds class files. HotSpot uses three built-in loaders arranged as a chain:

Loader	Name	Loads
Bootstrap	(shown as `null`)	The core of the platform, such as `java.lang.String`
Platform	`platform`	Other platform modules
Application	`app`	Your classes and the libraries on your class path

A loader asks its **parent** first, and only loads the class itself if the parent cannot. This **parent delegation** rule guarantees that nobody can replace `java.lang.String` with a fake version just by putting a class of that name on the class path.

The next program shows laziness and loaders in action.

File: `classloading/ClassLoadingDemo.java`

```
public class ClassLoadingDemo {

    static class Engine {
        static {
            System.out.println("  [Engine class is being initialized]");
        }
        static void start() {
            System.out.println("  Engine started");
        }
    }

    static {
        System.out.println("[ClassLoadingDemo class is being initialized]");
    }

    public static void main(String[] args) {
        System.out.println("main() begins");
        System.out.println("Loader of String:           " + String.class.getClassLoader());
        System.out.println("Loader of this class:       " +
         ClassLoadingDemo.class.getClassLoader().getName());
        System.out.println("Parent of application loader: "
                + ClassLoadingDemo.class.getClassLoader().getParent().getName());
        System.out.println("Before first use of Engine");
        Engine.start();
        System.out.println("After first use of Engine");
        Engine.start();
    }
}
```

Output

```
[ClassLoadingDemo class is being initialized]
main() begins
Loader of String:           null
Loader of this class:       app
Parent of application loader: platform
Before first use of Engine
  [Engine class is being initialized]
  Engine started
After first use of Engine
  Engine started
```

What to notice.

- `ClassLoadingDemo` is initialised first, before `main` runs, so its static block prints first.
- The nested `Engine` class is **not** initialised when `main` starts, nor when the line "Before first use of Engine" prints. Its static block runs only when `Engine.start()` is called for the first time, and runs **once**: the second call prints no initialisation message.
- `String.class.getClassLoader()` prints `null`: the bootstrap loader is represented as `null`. Our own class was loaded by `app`, whose parent is `platform`.

> **Interview Insight**
>
> "When does a class get loaded and initialised?" is a classic question. Strong answers mention laziness, the three phases, the loader hierarchy with parent delegation, and that static initialisation runs exactly once per loader. Frameworks such as Spring depend on this knowledge when they scan and load classes by name.

2.5 Executing Bytecode: Interpreter and JIT Compiler

Once a class is ready, the JVM's **execution engine** runs its bytecode. It has two ways to do so.

- The **interpreter** reads bytecode instructions one by one and performs each. It starts instantly but is slow per instruction.
- The **JIT (Just-In-Time) compiler** translates bytecode into real machine code for the current CPU while the program is running, and then the CPU runs that machine code directly at full speed.

HotSpot combines both in a design called **tiered compilation**:

1. A method starts out **interpreted**, and the JVM counts how often it is called and how often its loops repeat.
2. When a method becomes **hot** (used a lot), the JVM compiles it with a fast, lightly optimising compiler (**C1**) and then, if it stays hot, with a slower, aggressively optimising compiler (**C2**) that uses what it learned about how the program actually behaves (for example, which branch is always taken, or which concrete class a call always reaches).
3. If an assumption later turns out to be wrong, the JVM can **deoptimise** the code: throw away the machine code and fall back to the interpreter.

This is why a Java program often starts slowly and then speeds up: the **warm-up** period. It is also why a quick benchmark that runs a loop once tells you almost nothing about real performance (Chapter 36 returns to this).

> **Real-World Analogy**
>
> Think of a new employee following a written procedure line by line (the interpreter). After doing the same task hundreds of times, the employee memorises it and does it quickly without the manual (compiled code). If the process changes, they go back to the manual (deoptimisation). The analogy stops at one point: the JVM can compile and optimise while the program keeps running and uses live statistics, which a person cannot.

2.6 JVM Memory: A First Tour

The JVM organises memory into **runtime data areas**. This is the vocabulary you need for the rest of the book.

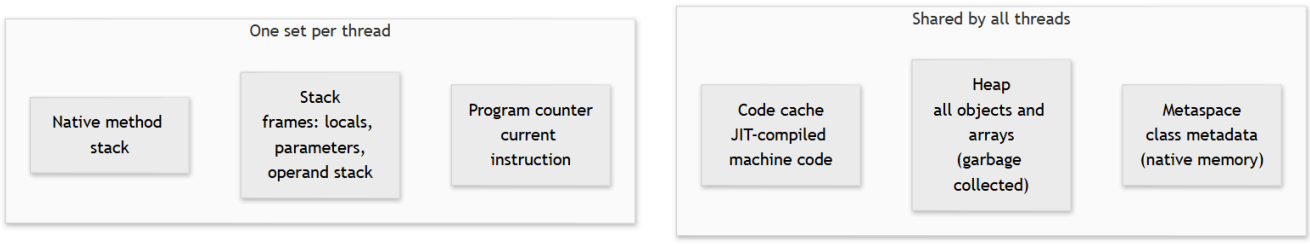


Figure 2.4: Main JVM memory areas

- **Heap.** Every object (`new Counter()`) and every array lives on the heap. It is shared by all threads and managed by the **garbage collector**, which reclaims objects that nothing can reach any more. The heap is usually the largest area; its maximum size is set with `-Xmx` (for example `-Xmx512m`). If no flag is given, the JVM picks a default based on the machine's memory, typically a quarter of physical RAM.
- **Stack.** Each thread has its own stack. Every method call pushes a **stack frame** holding that call's local variables, parameters and operand stack, and the frame is removed when the method returns. Stacks are small and fast. Their size is set with `-Xss`. Too many nested calls (usually infinite recursion) causes `StackOverflowError`.
- **Metaspace.** Stores class metadata: method and field descriptions, the constant pool, and so on. It lives in native (non-heap) memory and grows as classes load. (In Java 7 and earlier this was called PermGen, a fixed-size area that caused many out-of-memory errors; Metaspace replaced it in Java 8.)
- **Code cache.** Holds machine code produced by the JIT.
- **Program counter** and **native method stack** track the current instruction and calls into non-Java (native) code.

The practical difference between stack and heap is shown by this program. A method receives a copy of a primitive value but a copy of a *reference* to an object:

File: `memory/StackAndHeap.java`

```
public class StackAndHeap {

    static class Counter {
        int value;
    }
```

```
    static void increment(int number, Counter counter) {
        number++;          // changes only the local copy on this method's stack frame
        counter.value++;   // changes the object on the heap, shared by every reference    }
             static int depth = 0;             static void recurseForever() {        depth++;
        recurseForever();    }                  public static void main(String[] args) {
        int number = 10;        Counter counter = new Counter();        increment(number, counter);
        System.out.println("number        = " + number);
        System.out.println("counter.value = " + counter.value);                try {
            recurseForever();        } catch (StackOverflowError e) {
            System.out.println("Caught StackOverflowError after many frames: " + (depth > 1000));
        }     }     }
```

Output

```
number        = 10
counter.value = 1
Caught StackOverflowError after many frames: true
```

Execution flow. `main` creates `number` (a primitive, stored in `main`'s stack frame) and a `Counter` object (stored on the heap; `main` holds only a reference to it). When `increment` runs it gets its own frame. `number++` changes `increment`'s own copy, so `main`'s `number` stays 10. `counter.value++` follows the reference to the *same* heap object, so the change is visible afterwards: 1. This is the heart of Java's **pass-by-value** behaviour, which Chapter 6 explains fully.

The last part provokes a `StackOverflowError` by recursing without end. Each call adds a frame until the thread's stack is exhausted. The program catches the error only to demonstrate it. In real code you fix the recursion rather than catching the error. We print whether more than 1,000 frames were created instead of the exact depth, because the exact number depends on the machine and JVM settings.

> **Warning**
>
> The words **stack** and **heap** here are *memory areas*, not the data structures you will meet in Chapters 11 and 51 (a "stack" is also a last-in-first-out collection). The names are shared on purpose: a method call stack really does behave last-in-first-out. Do not confuse the two meanings.

Chapter 16 returns to memory in depth: object layout, generations, garbage collectors, heap dumps and tuning.

2.7 Platform Independence: What It Really Means

"Write once, run anywhere" is true for **bytecode**: the same `Add.class` runs on Windows, Linux and macOS. The JVM, however, is **platform specific**: Oracle, Eclipse and others publish a different JVM build for each operating system and CPU family. The JVM acts as a translator between portable bytecode and the local machine.

A few limits to the promise:

- Native libraries (`.dll`, `.so`) and calls into the operating system are platform dependent.

- File paths, line endings and default character sets differ between systems, so careless code can behave differently across them.
- Behaviour that depends on thread scheduling or timing can vary.

Another consequence of this design is that the JVM is not only for Java. Other languages, including **Kotlin**, **Scala** and **Groovy**, compile to the same bytecode and run on the same JVM, and they can call Java libraries such as Spring.

2.8 Java Versions, Releases and LTS

Since 2017, a new Java version has been released every six months (March and September). Most releases are supported only until the next one. Certain releases are designated **LTS (Long-Term Support)**: **8, 11, 17, 21 and 25**. Organisations normally run LTS versions in production because vendors provide fixes for many years.

Features arrive through **JEPs** (JDK Enhancement Proposals). A new feature may first appear as a **preview** feature, which you must explicitly enable with `--enable-preview`, so that the community can try it before the design is finalised. Do not use preview features in production code.

This is how this book treats versions:

Version	Status in this book	What to know
Java 8	Legacy reference	Lambdas and streams (Chapter 13). Still found in older systems.
Java 11	Legacy reference	`var` in lambdas, HTTP client, single-file launch
Java 17	Baseline	Records, sealed classes, text blocks, switch expressions
Java 21	**Primary**	Pattern matching, record patterns, virtual threads
Java 25	Upgrade reference	Latest LTS; differences noted in Chapter 14

You can confirm which Java version a program is running on with a few lines:

File: `version/ShowVersion.java`

```
public class ShowVersion {
    public static void main(String[] args) {
        System.out.println("Java feature version: " + Runtime.version().feature());
        System.out.println("Java vendor: " + System.getProperty("java.vendor"));
    }
}
```

Output

```
Java feature version: 21
Java vendor: Eclipse Adoptium
```

(The vendor line shows the JDK build used to produce this book; yours may show a different vendor.)

2.9 Common Mistakes

Common Mistake

`UnsupportedClassVersionError`. You compile with Java 21 but run with an older runtime (for example Java 17): `... has been compiled by a more recent version of the Java Runtime (class file version 65.0), this version of the Java Runtime only recognizes class file versions up to 61.0`. *Cause:* the class file's major version (65) is newer than the runtime understands (61). *Fix:* run with Java 21 or newer, or compile for the older target (`javac --release 17`). Check with `java -version` and `javac -version`, which can report different versions if several JDKs are installed.

Common Mistake

Two JDKs, wrong one on the PATH. `javac -version` prints 21 but `java -version` prints 17 (or the reverse). *Cause:* the PATH finds a different installation for each command. *Fix:* set `JAVA_HOME` to the JDK you want, put `%JAVA_HOME%\bin` (Windows) or `$JAVA_HOME/bin` (macOS/Linux) first on the PATH, and open a new terminal.

Common Mistake

Assuming static initialisation happens at start-up. A static block that is expected to run "when the program starts" never runs because the class is never used. *Fix:* remember that initialisation is lazy and triggered by first active use. Do not rely on a class being initialised unless your code uses it.

2.10 Best Practices

Best Practice

- **Develop and run on the same major version** that you deploy. Surprises hide in version differences.
- **Prefer LTS releases** for production, and plan upgrades between LTS releases rather than staying on one for a decade.
- **Use `--release N` when compiling** if you must produce class files for an older runtime. It also restricts you to that version's standard library, which `-source` and `-target` alone do not.
- **Do not depend on JIT behaviour or warm-up** in correctness. Use JIT knowledge for performance work only, and always measure.
- **Never change the standard library by adding classes with `java.*` names** to your class path; parent delegation prevents it and the attempt signals a design problem.

2.11 Practice

1. **Compile for an older release.** Compile `HelloWorld` with `javac --release 17 HelloWorld.java`, then run `javap -v HelloWorld.class` and find the `major version` line. What number do you see? Repeat with `--release 11`.
2. **Read bytecode.** Write a method `static int triple(int x) { return x * 3; }`, compile it and run `javap -c -p`. Which instruction does the multiplication? Which instruction pushes the constant 3?
3. **Initialisation order.** Create two classes, `A` and `B`, each with a static block that prints its name. In `main`, print "start", then create a `B`, then an `A`, then another `B`. Predict the exact output before you run it.
4. **Find the loader.** Print the class loader of `java.util.ArrayList`, of your own class, and of `java.sql.Connection`. Explain each result using the three-loader table.
5. **Break the stack.** Write a recursive method that counts how deep it goes before `StackOverflowError`. Run it with `java -Xss512k` and `java -Xss4m`. What changes, and why is the number different between two runs on different machines?
6. **Challenge: JDK versus JRE.** Look inside your JDK's `bin` folder and list the tools. Pick three that you do not recognise and look up what each does in the official documentation.

2.12 Interview Questions and Answers

Q1. What is the difference between JDK, JRE and JVM? The JVM executes bytecode. The JRE is the JVM plus the standard class libraries. The JDK is the JRE plus development tools such as `javac`. Developers need the JDK; running an already built application needs only the runtime.

Q2. What is bytecode, and what is a class file? Bytecode is the instruction set of the JVM, an abstract stack-based machine. A class file is the binary file that holds one class: its name, parent, constant pool, fields, method bytecode and a version number. The compiler generates it and the JVM loads it.

Q3. Describe the class-loading process. The class is loaded by a class loader, then linked (verification of bytecode safety, preparation of static fields with default values, and resolution of symbolic references), and finally initialised by running static initialisers. This happens lazily, the first time the class is needed.

Q4. What is parent delegation in class loading, and why does it exist? A class loader first asks its parent to load a class and only tries itself if the parent fails. It ensures core classes always come from the trusted bootstrap loader, so application code cannot replace `java.lang.String`, and it avoids loading the same class twice.

Q5. What is the JIT compiler and why does the JVM need both an interpreter and a JIT? The interpreter starts immediately but is slow; the JIT compiles frequently used bytecode to optimised machine code at run time. Using both gives fast start-up and high steady-state speed. HotSpot's tiered compilation uses C1 for quick compilation and C2 for heavily optimised code, with deoptimisation as a safety net.

Q6. Where are objects, local variables and class metadata stored? Objects and arrays are on the heap. Local variables and method call state are in stack frames, one stack per thread (a local variable that is a reference stores the reference on the stack, and the object it points to on the heap). Class metadata is in Metaspace, which uses native memory.

Q7. What is the difference between `StackOverflowError` and `OutOfMemoryError`? `StackOverflowError` happens when a thread's stack is exhausted, usually from infinite or very deep recursion. `OutOfMemoryError`

happens when the JVM cannot allocate memory, for example a full heap, Metaspace or native memory. They have different causes and different fixes. Increasing `-Xss` can delay a stack overflow but does not repair runaway recursion.

Q8. (Scenario) The application runs fine on your laptop but fails on the server with `UnsupportedClassVersionError`. What happened? The application was compiled on a newer JDK (major version 65 for Java 21) than the runtime on the server. Run `java -version` on the server, then either upgrade the server's runtime or compile with `--release` set to the server's version.

Follow-up: *Is Java compiled or interpreted?* Both. Source is compiled to bytecode by `javac`, and at run time the JVM interprets that bytecode and JIT-compiles hot code to machine code.

2.13 Chapter Summary

- The **JVM** runs bytecode, the **JRE** adds the class libraries, and the **JDK** adds development tools.
- `javac` turns source into **class files**; `javap` lets you read their bytecode. A class file carries a major version (65 for Java 21).
- The JVM loads classes **lazily**, then **links** them (verify, prepare, resolve) and **initialises** them once, using a chain of class loaders with **parent delegation**.
- The execution engine **interprets** bytecode and uses **tiered JIT compilation** (C1, then C2) to compile hot code, with **deoptimisation** when assumptions fail.
- The main memory areas are the **heap** (objects), per-thread **stacks** (call frames), **Metaspace** (class metadata) and the **code cache**.
- Bytecode is portable; JVM builds are platform specific.
- New Java versions arrive every six months. LTS releases are 8, 11, 17, 21 and 25; this book teaches on Java 21.

2.14 Key Terms and Revision Notes

Term	Meaning
Bytecode	Portable instructions for the JVM, stored in `.class` files
Class file	Binary file for one class; begins with `CA FE BA BE` and has a major version
Constant pool	Table of names and literals inside a class file
Class loader	Finds and loads class files; bootstrap, platform and application loaders
Parent delegation	A loader asks its parent first before loading itself
Linking	Verification, preparation and resolution
Initialisation	Running static blocks and static initialisers, once, on first active use
Interpreter	Executes bytecode instruction by instruction
JIT compiler	Compiles hot bytecode to machine code at run time (C1, C2)
Deoptimisation	Discarding compiled code and returning to interpretation
Heap	Memory for objects and arrays, garbage collected
Stack frame	Per-call record of locals, parameters and operand stack
Metaspace	Native memory for class metadata
LTS	Long-Term Support release: 8, 11, 17, 21, 25

Quick revision checklist.

- ☐ I can draw the JDK/JRE/JVM nesting from memory.
- ☐ I can list the lifecycle stages from source file to execution.
- ☐ I can read the four instructions behind `a + b` in `javap -c` output.
- ☐ I can explain why `Engine`'s static block ran only once and only on first use.
- ☐ I can say where an object, a local variable and a class definition live in memory.

Where we go next. Chapter 3 starts writing real Java: variables, data types, operators, and the surprises (overflow, floating-point rounding, autoboxing) that trip up beginners and interviewees alike.

References for this chapter. Oracle, *The Java Virtual Machine Specification, Java SE 21 Edition* (class file format, class loading, runtime data areas); Oracle, *The Java Language Specification, Java SE 21 Edition* (class initialisation); OpenJDK documentation for the `javac`, `javap` and `java` tools; Oracle Java SE support roadmap for LTS releases.